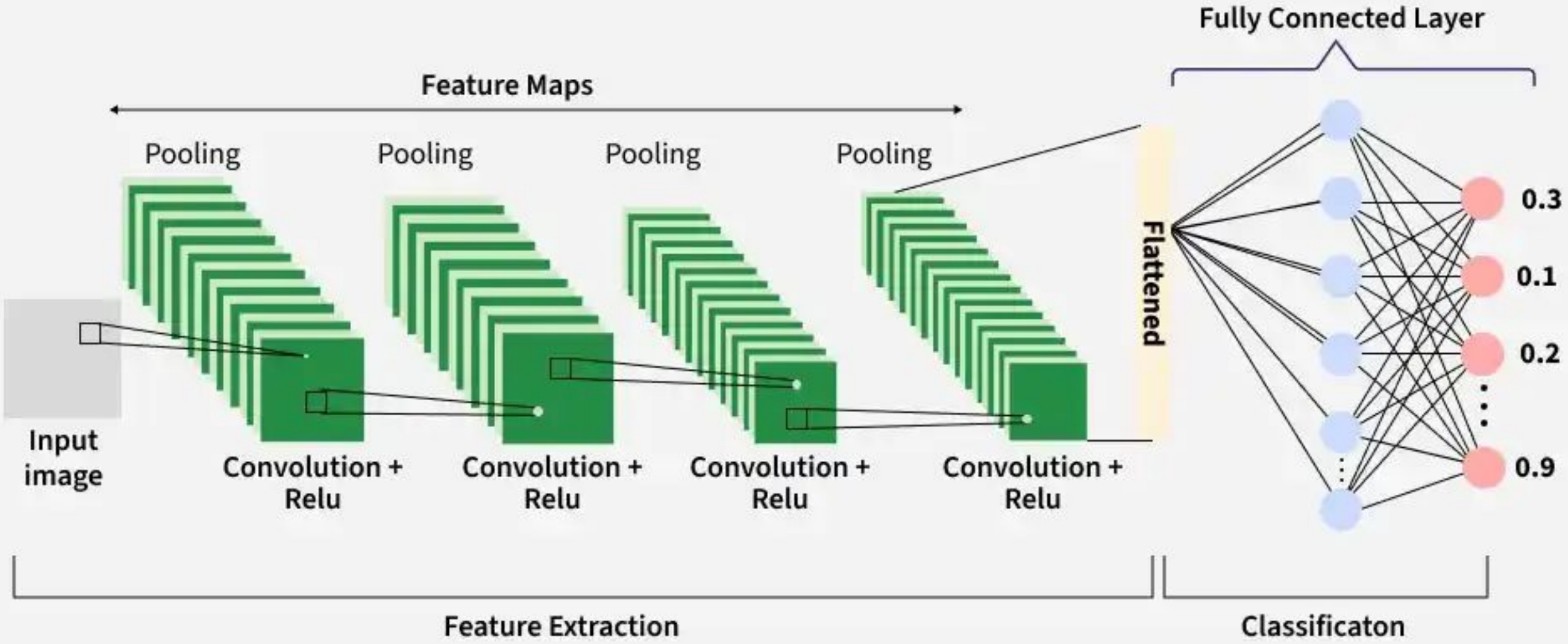
Convolutional Neural Networks

Convolutional Neural Networks



Convolutional Neural Networks

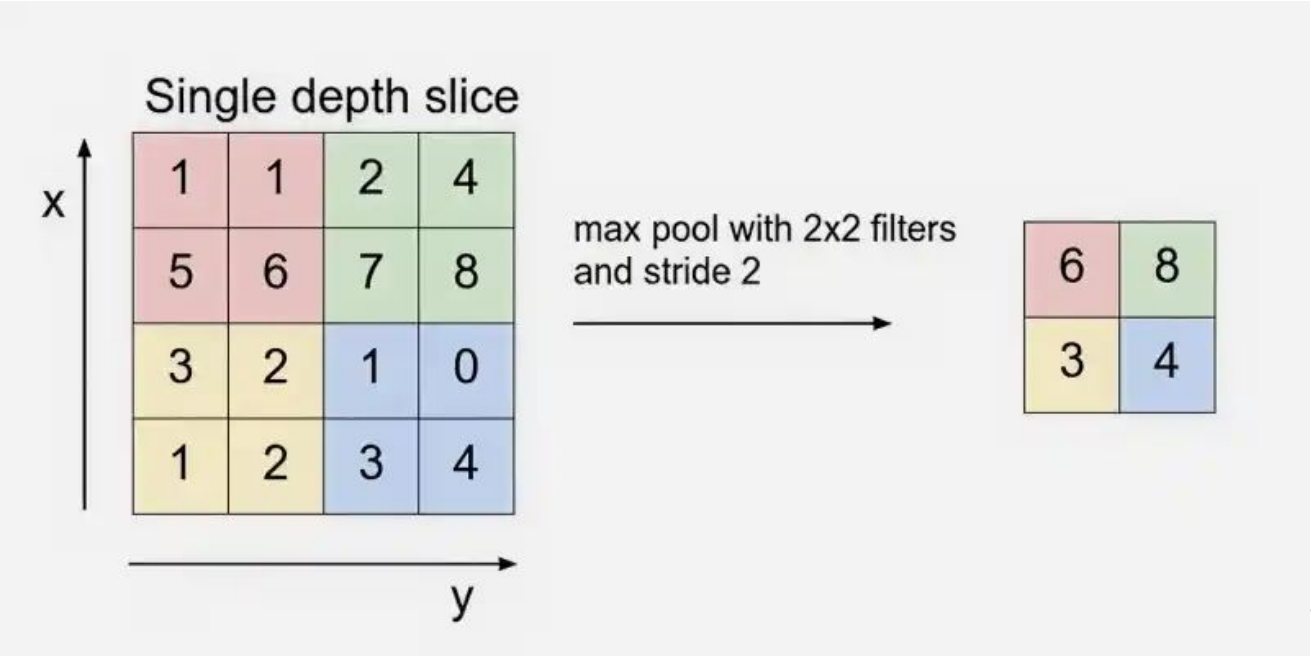
- Inspired by the human visual system and are widely used in computer vision tasks.
- They are designed to process structured grid-like data, especially images by capturing spatial relationships between pixels.
- Automatically learn hierarchical features through convolution operations, from simple edges and textures to complex shapes and objects.
- Detect objects at different positions within an image, ensuring robustness to spatial variations.
- Reduce computational complexity by processing local regions instead of the entire image at once.

CNN Layers

- **Input layer:** raw image data
  - typically a 3D volume (width × height × depth)
- **Convolutional Layer**
  - Extract important features from the input data
  - It applies a set of learnable filters (kernels) that slide over the image and compute the dot product between the filter weights and corresponding image patches, producing feature maps.
    - Uses small filters (e.g., 2×2, 3×3, 5×5) to scan the input image.
    - Generates feature maps that capture patterns such as edges, textures and shapes.
- **Activation Layer**
  - introduces non-linearity into the network by applying an element-wise activation function to the output of the convolution layer.
    - ReLU, Tanh and Leaky ReLU
  - Applied element-wise to the feature maps.
  - The output dimensions remain unchanged (e.g., 32 × 32 × 12).

CNN Layers

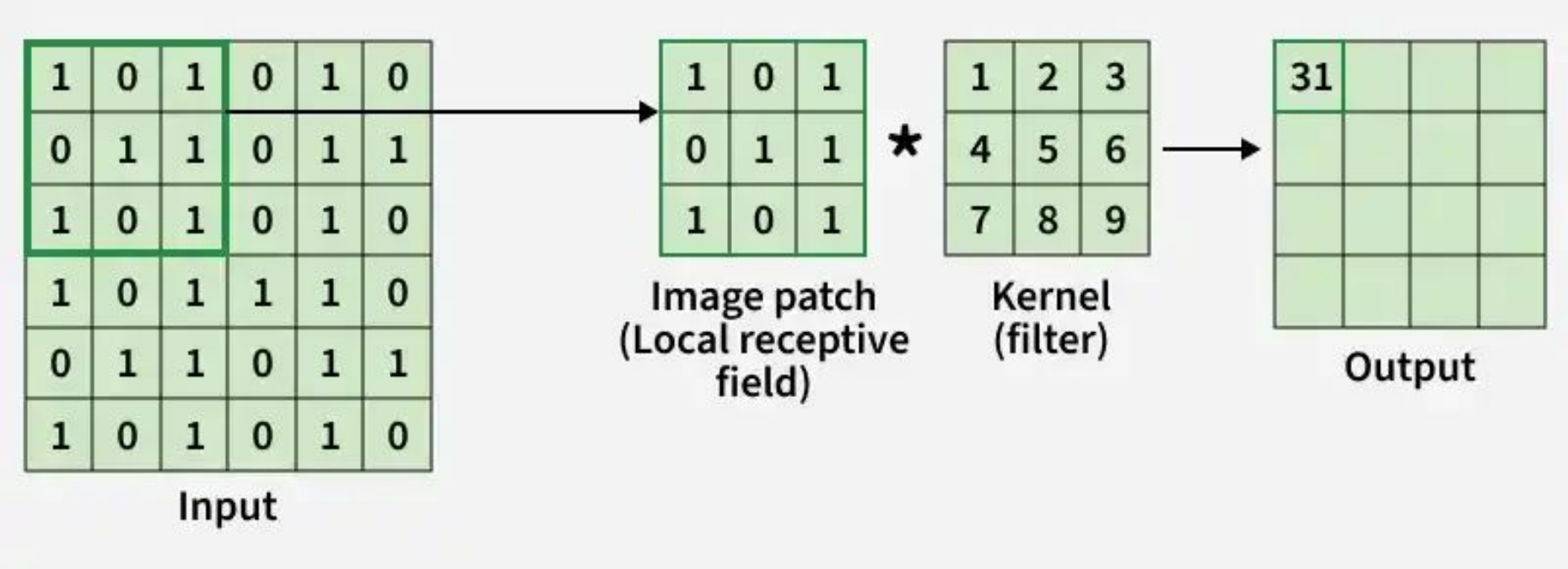
- **Pooling Layer**
  - Used to reduce the spatial dimensions of the feature maps, making computation faster, reducing memory usage and helping to prevent overfitting.
    - Max Pooling and Average Pooling
    - Reduces width and height while keeping depth unchanged.



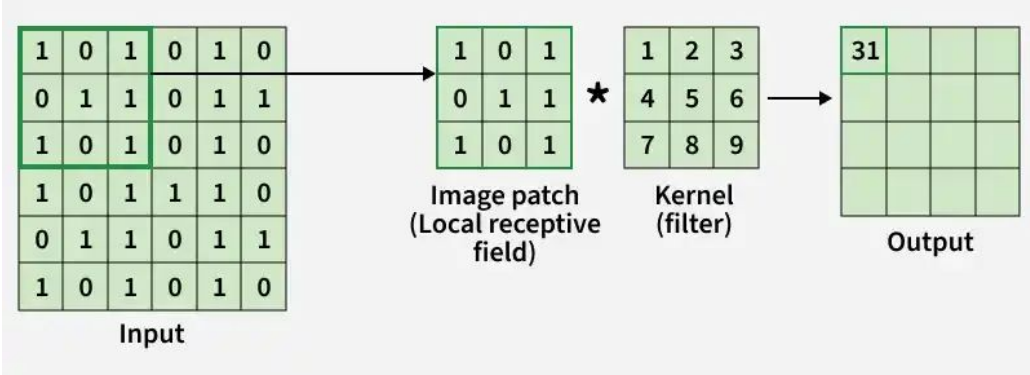
CNN Layers

- **Flattening**
  - converts the multi-dimensional feature maps into a one-dimensional vector after convolution and pooling.
  - This vector is then passed to the fully connected layer for classification or regression.
    - *Flattening 16 × 16 × 12 results in a vector of size 3072 (16 × 16 × 12).*
- **Fully Connected Layer**
  - Performs high-level reasoning using extracted features and produces the final classification scores.
    - *The 3072-length vector is connected to neurons for classification*
- **Output Layer**
  - converts final scores into probabilities using activation functions like Sigmoid (binary classification) or Softmax (multi-class classification).

How CNN Works

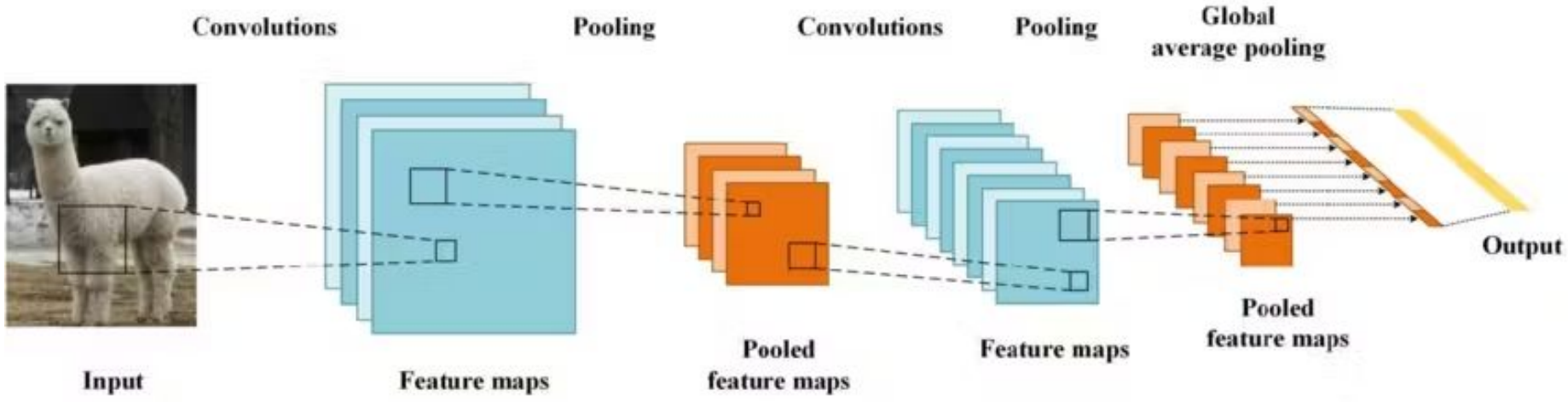


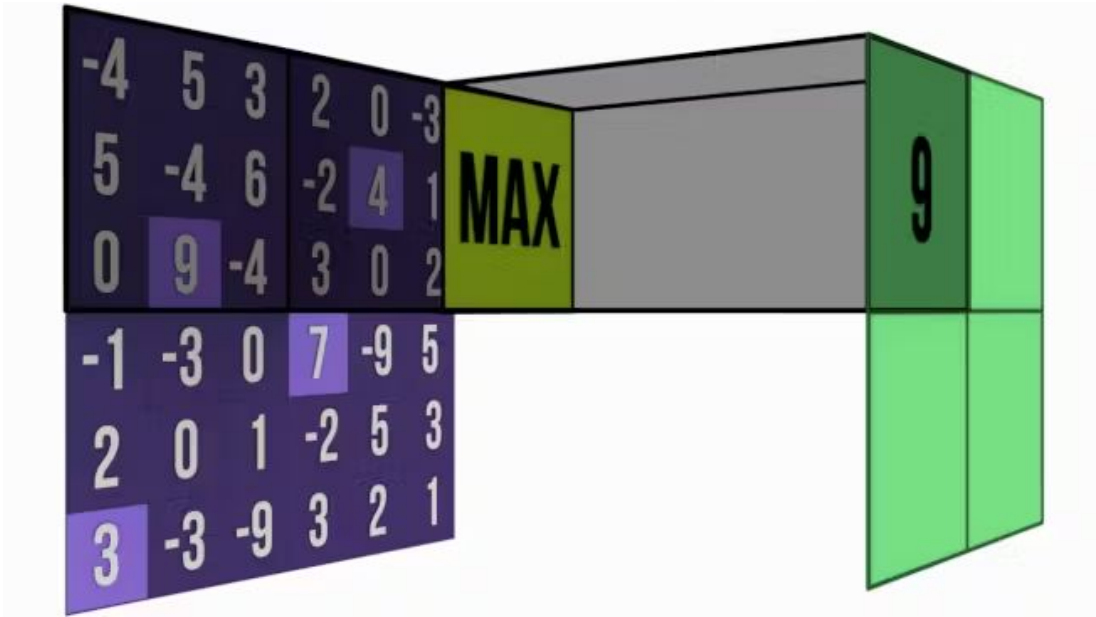
How CNN Works



- **The Input (6×6)**
  - The 6×6 grid of 0s and 1s represents a **binary feature map or grayscale image patch** — each cell is one pixel value (or activation). The highlighted 3×3 top-left region (darker green border) is the **current position** of the sliding kernel.
- **Local Receptive Field (3×3)**
  - This is the **extracted 3×3 sub-region** from the input that the kernel is currently looking at
- The Kernel / Filter (3×3)
  - These are the **learned weights**: the parameters the network trains during backpropagation. This particular kernel has ascending values, so it weights the bottom-right of each patch more heavily.
- How 31 is calculated:
  - Sum of element-wise multiplication
- Each output cell answers: "How much does this 3×3 patch of the input resemble what this kernel is looking for?"
- A high value means strong match, low value means weak match.

Pooling Operation





Parameter Sharing

- Parameter sharing in CNNs is a technique where the same set of weights (filter/kernel) is applied across different spatial locations of an input, enabling efficient feature detection, such as edge detection, regardless of position. This approach drastically reduces the total number of trainable parameters compared to fully connected networks, preventing overfitting and lowering memory usage.
- **Weight Replication:**
  - In a convolutional layer, all neurons within a single feature map share the same weights. If a filter learns to detect a specific feature (like a horizontal edge), it can detect that feature anywhere in the input image.
- **Reduced Memory Footprint:**
  - Instead of separate weights for every connection, a filter uses only unique weights, plus a bias, regardless of the input size.
- **Translation Invariance:**
  - Because the same filter slides across the image (convolution operation), the network is robust to the location of features, enabling the detection of an object even if it moves slightly.

Parameter Sharing

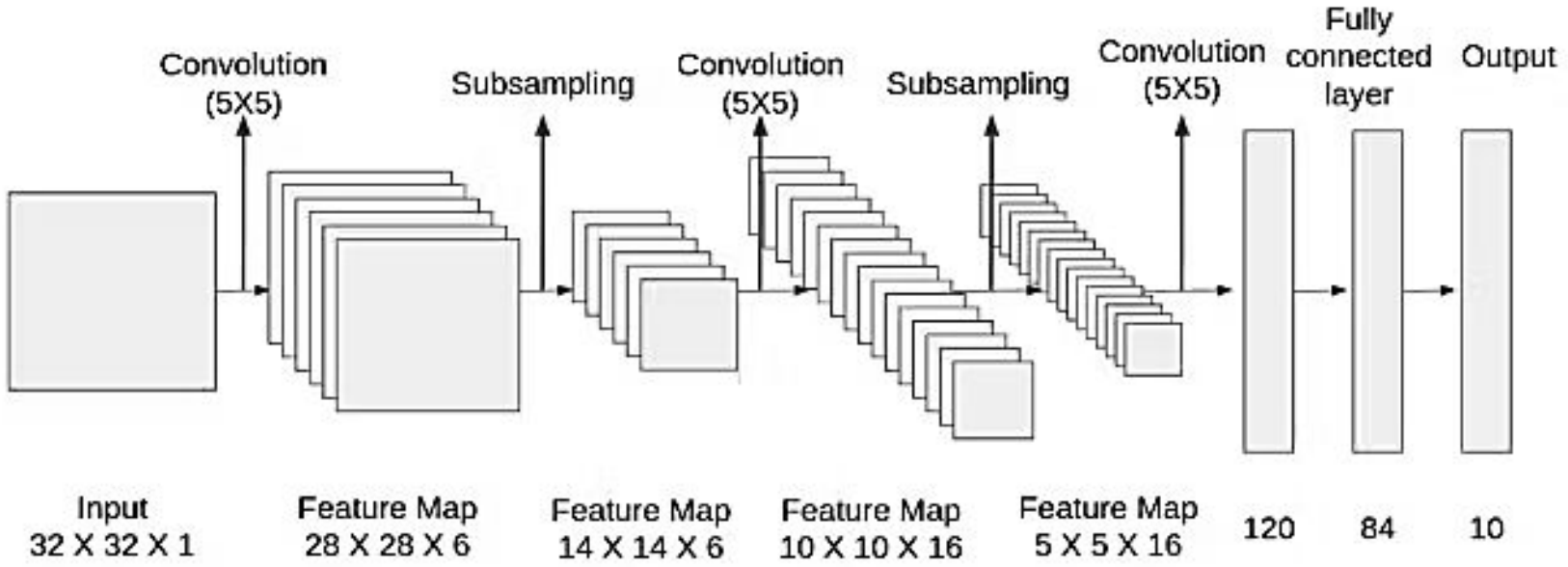
- **Computational Efficiency:**
  - Parameter sharing requires fewer weight updates during backpropagation, resulting in faster training times and lower computational costs.
- One kernel, learned once, applied everywhere.
- The same weights are reused at every spatial position in the input.
- **The Problem it Solves:**
  - Imagine you want a neuron to detect a vertical edge. In a fully connected layer, you'd need a separate set of weights for every possible location that edge could appear:
    - "vertical edge at top-left" → unique weights
    - "vertical edge at top-right" → different unique weights
    - "vertical edge at center" → yet another set of weights
  - An edge is an edge regardless of where it sits in the image. Parameter sharing encodes this common sense directly into the architecture.

Architectures and Variants of CNNs

- **LeNet-5 (**Yann LeCun et al. 1998)
- **AlexNet (**Alex Krizhevsky et al. 2012)
- **VGGNet (**Karen Simonyan and Andrew Zisserman 2014)
- **ResNet (**Kaiming He et al. 2015)
-

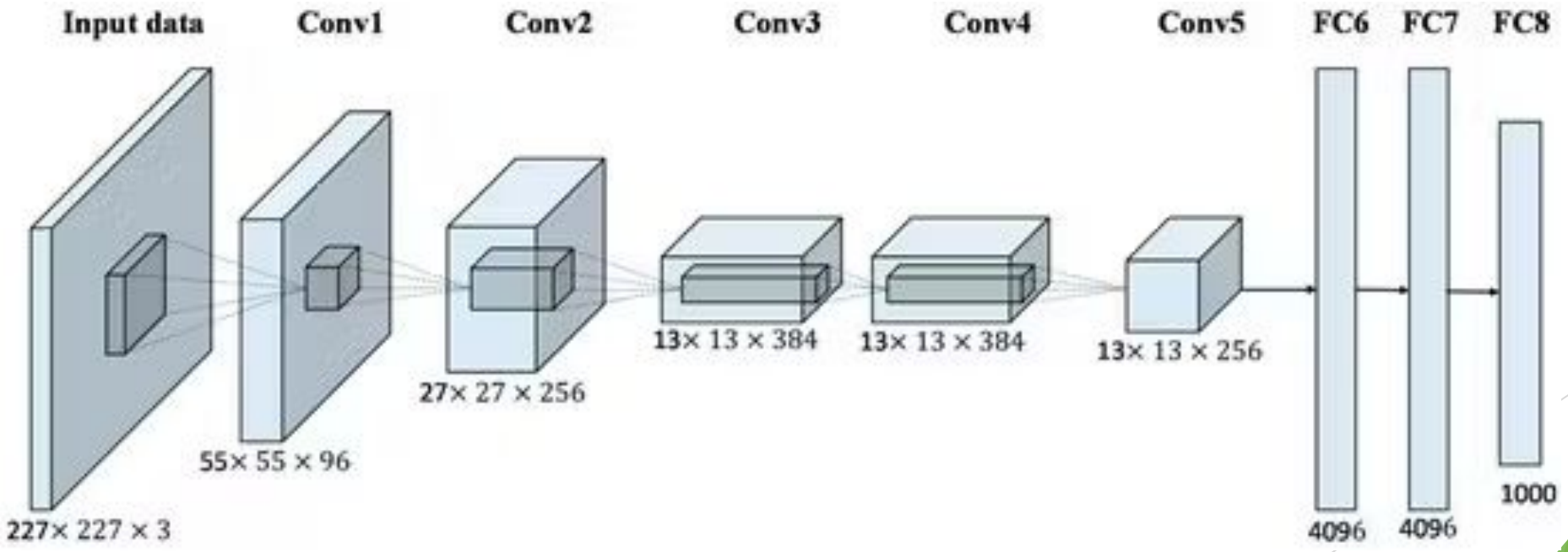
LeNet-5

- One of the pioneering CNN architectures. It was designed for handwritten digit recognition and consisted of two convolutional layers, followed by average pooling, fully connected layers, and a softmax output layer. LeNet-5's lightweight architecture and efficient use of shared weights laid the foundation for modern CNNs, influencing subsequent developments in the field.



AlexNet

- made significant strides in image classification. AlexNet featured a deeper architecture with multiple convolutional layers, introduced the concept of ReLU activations, and employed techniques like dropout for regularization.



AlexNet

- Layer 1 → edges, color gradients, cornersLayer
- 2 → textures, simple shapes (circles, curves)Layer
- 3 → object parts (eyes, wheels, windows)Layer
- 4 → object components (faces, car fronts)Layer
- 5 → whole objects (cat, car, chair)

AlexNet

Layer	Type	Output Shape	Key Details
Input	—	227×227×3	RGB image
Conv1	Conv + ReLU + MaxPool	55×55×96	11×11 kernel, stride 4
Conv2	Conv + ReLU + MaxPool	27×27×256	5×5 kernel, padding 2
Conv3	Conv + ReLU	13×13×384	3×3 kernel, padding 1
Conv4	Conv + ReLU	13×13×384	3×3 kernel, padding 1
Conv5	Conv + ReLU + MaxPool	13×13×256	3×3 kernel, padding 1
FC6	Fully Connected	4096	Dropout applied
FC7	Fully Connected	4096	Dropout applied
FC8	Fully Connected	1000	Softmax output (ImageNet classes)

AlexNet

- ReLU activations used instead of tanh/sigmoid; much faster training
- Dropout (0.5) in FC6 & FC7 prevents overfitting
- **Local Response Normalization (LRN)** after Conv1 and Conv2
- Overlapping MaxPooling: 3×3 pool with stride 2
- Data Augmentation: random crops, flips, color jitter during training